

Documentatie - Proiect ISP

APLICATIE PENTRU GESTIONAREA VOLUNTARILOR LA ONG

Nume: Cosmulete Ion-Cosmin

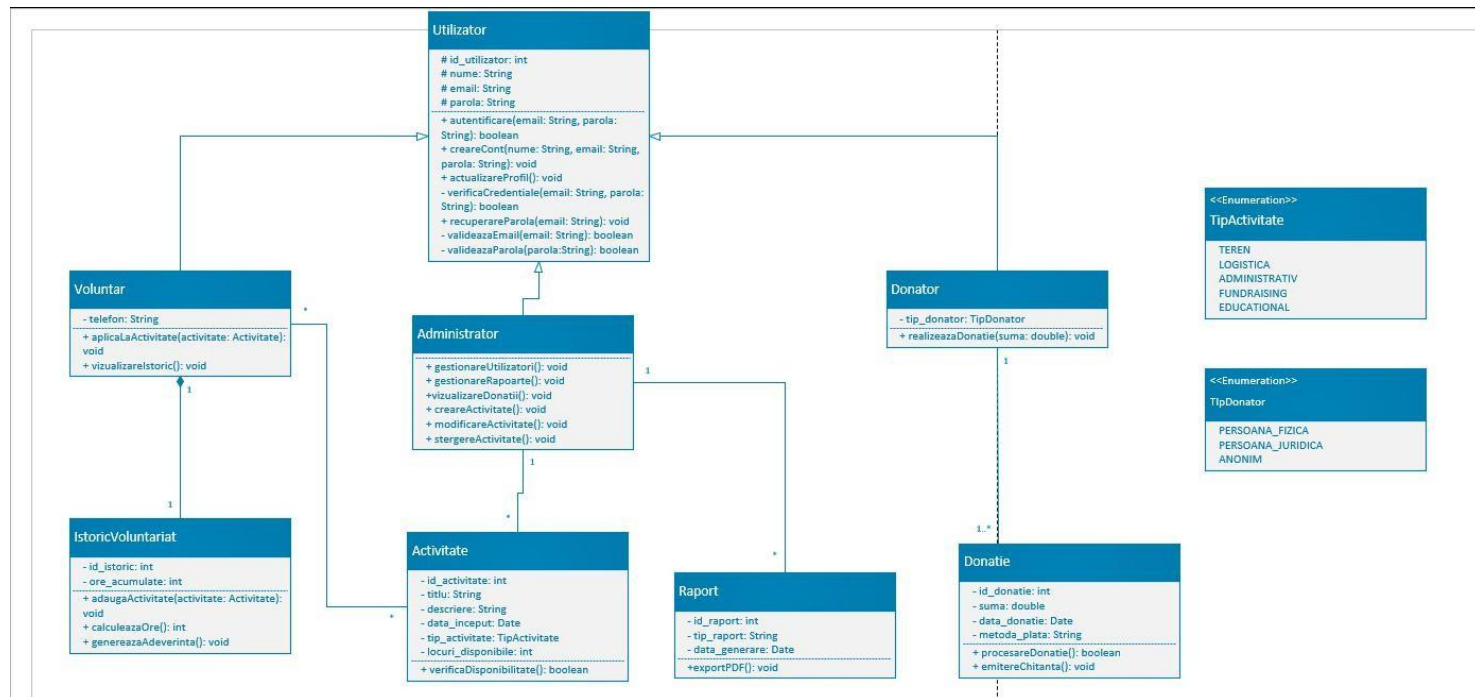
Grupa: 332AA

Descriere:

Proiectul reprezintă un sistem software dezvoltat în Java, destinat administrării activităților unui ONG. Aplicația permite gestionarea a trei tipuri principale de utilizatori: Administratori (care creează activități și rapoarte), Voluntari (care se pot înscrie la activități și pot genera adeverințe cu istoricul de voluntariat) și Donatori (care pot realiza donații pentru a susține ONG-ul). Pentru a asigura calitatea codului și respectarea arhitecturii, proiectul este organizat în două pachete distincte (models și tests). Funcționalitățile critice sunt testate automatizat folosind framework-ul JUnit. De asemenea, pentru exportul rapoartelor și adeverințelor a fost integrată biblioteca externă iTextPDF.

1. Partea I - Lucru în echipa

1.1. Diagrama de clase



Descrierea modelului:

Diagrama de clase modelează structura sistemului și conține cele trei tipuri de relații obligatorii studiate:

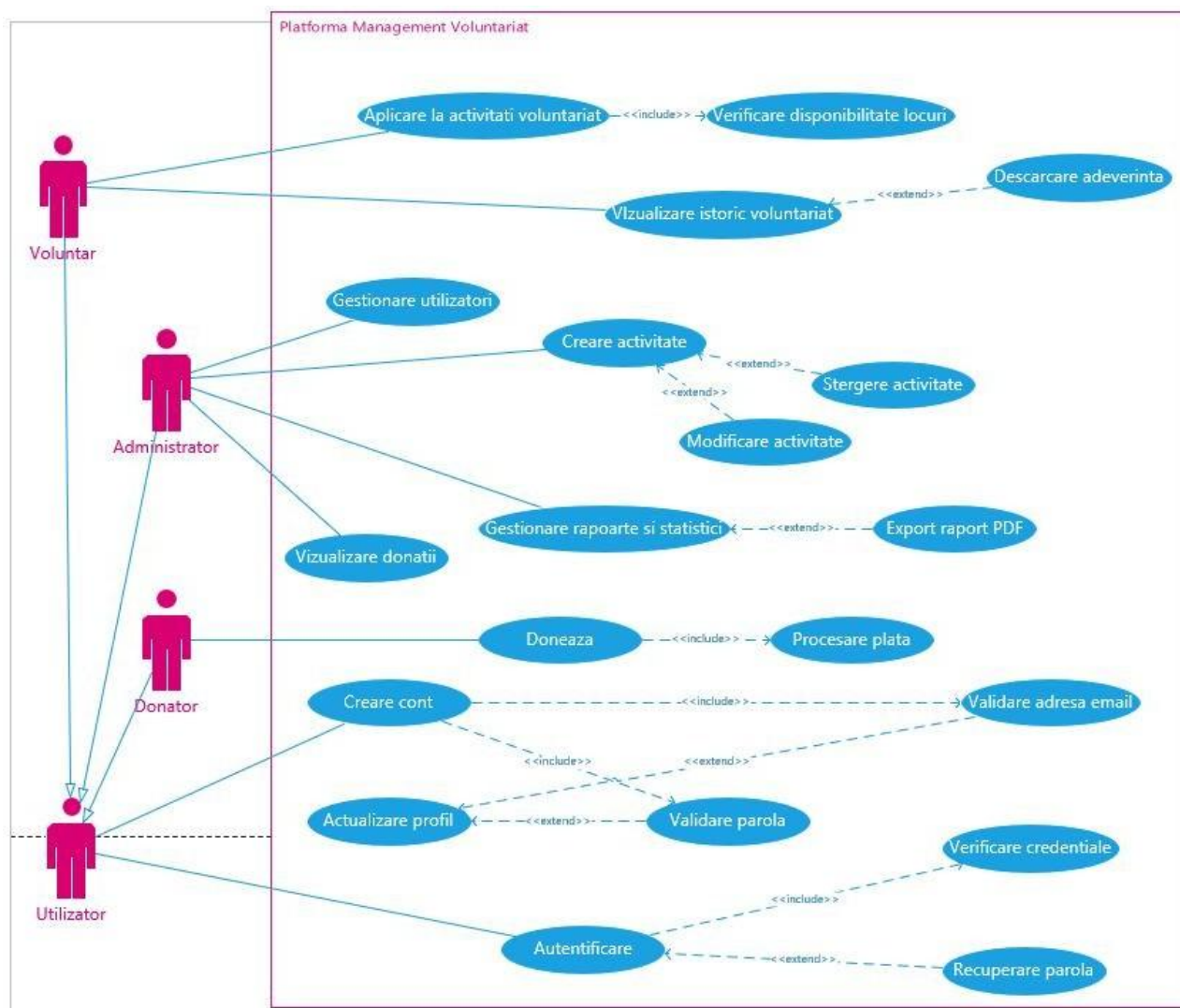
-Generalizare (Moștenire): Clasa abstractă Utilizator este extinsă de clasele derivate Administrator, Voluntar și Donator. Toate moștenesc atributele de bază (id_utilizator, nume, email, parola) și comportamentele comune (autentificare, creare cont).

-Compoziție: Există o relație strânsă între Voluntar și IstoricVoluntariat (1 la 1), istoricul fiind instanțiat direct în constructorul voluntarului, așadar ciclul de viață al istoricului depinde de cel al voluntarului.

-Asociere: Clasa Administrator este asociată cu gestionarea claselor Activitate și Raport. Relații de asociere există între Donator și Donatie (un donator are o listă de donații asociate, 1 la 1..*), dar și între Voluntar și Activitate (un voluntar poate participa la mai multe activități, iar o activitate are o listă de voluntari).

-Enumerări: Sunt folosite enums precum TipActivitate (TEREN, LOGISTICA, etc.) și TipDonator.

1.2 Diagrama cazurilor de utilizare



Descrierea modelului:

Diagrama surprinde interacțiunea actorilor cu sistemul:

-**Actori:** Utilizator (actor general), Voluntar, Administrator, Donator.

-**Cazuri de utilizare principale:** Autentificare, Creare cont (comune tuturor).

Administratorul are acces la: Creare/Modificare/Stergere activitate, Gestionare utilizatori și Gestionare rapoarte. Voluntarul poate: Aplica la activități, Vizualizare și descărcare adeverinta. Donatorul poate realiza donații.

-**Relații <<include>> și <<extend>>:**

* Include: "Donează" include "Procesare plată".

"Aplicare la activități voluntariat" include obligatoriu "Verificare disponibilitate locuri".

*Extend: "Creare activitate" poate fi extinsă opțional de "Modificare activitate" sau "Ștergere activitate".

"Vizualizare istoric voluntariat" poate fi extins de "Descărcare adeverință" .

2. Partea II - Lucru individual

2.1. Diagrama de activități

Descrierea modelului:

Această diagramă ilustrează procesul prin care un Administrator creează și adaugă o nouă activitate în platforma ONG-ului.

Fluxul principal (Inițierea): Procesul începe (Nodul inițial) când **Administratorul** solicită crearea unei activități noi („Solicitare creare activitate nouă”).

Sistemul preia cererea și afișează interfața corespunzătoare („Afișare formular creare Activitate”).

Administratorul completează formularul cu informațiile necesare („Introducere date activitate”).

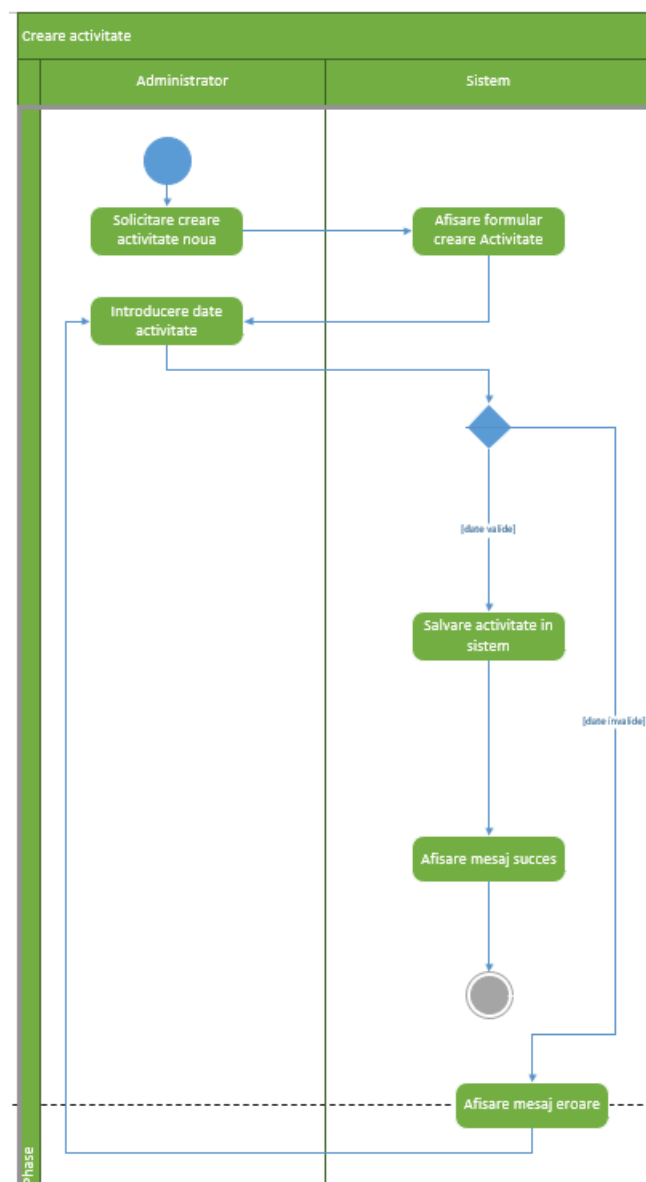
Decizia și Validarea (Nodul de decizie): Sistemul preia datele introduse și le validează, moment în care fluxul se desparte în două direcții posibile:

Ramura de succes [date valide]: Dacă datele respectă regulile de validare (ex: locuri > 0, ID unic, format corect), Sistemul salvează activitatea în baza de date („Salvare activitate în sistem”), afișează un mesaj de confirmare („Afișare mesaj succes”), iar procesul se încheie cu succes (Nodul final).

Ramura de eroare/alternativă [date invalide]: Dacă datele introduse sunt greșite sau incomplete (ex: locuri negative, câmpuri goale), Sistemul afișează un avertisment („Afișare mesaj eroare”) și **redirecționează fluxul înapoi** la pasul de completare a formularului („Introducere date activitate”), oferind Administratorului posibilitatea de a corecta informațiile.

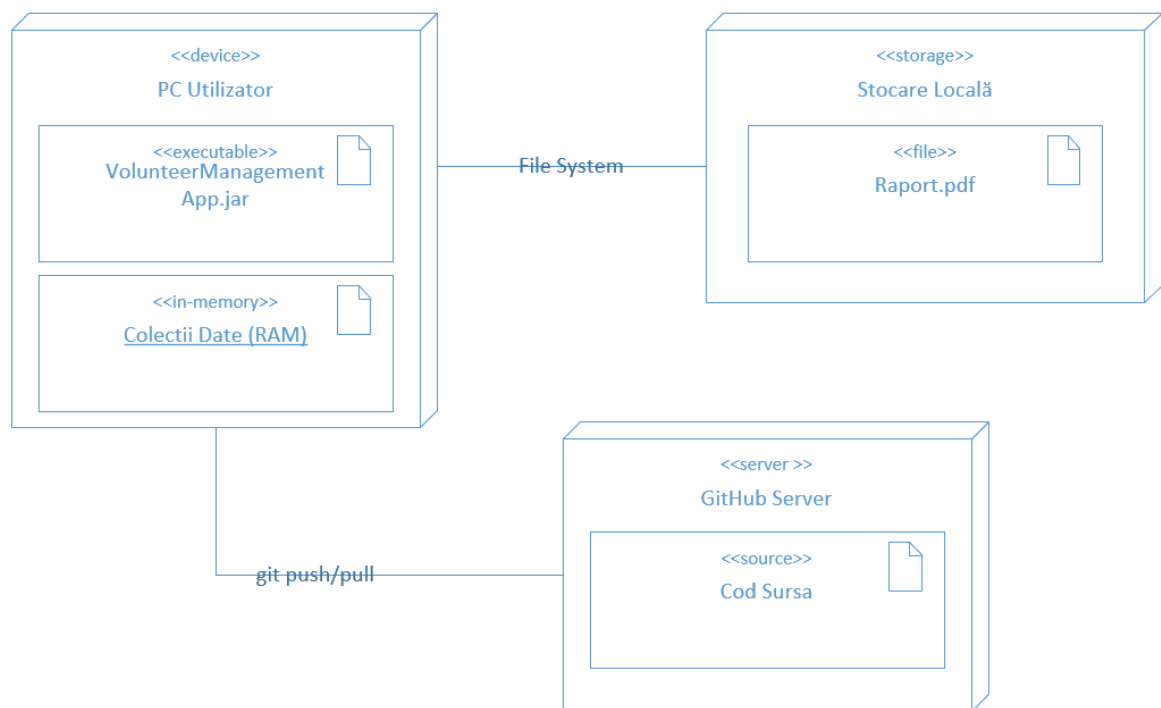
Precizare: Acest flux a fost testat unitar în clasa CreareActivitateTest.java (folosind JUnit). Pentru validarea corectitudinii acestui flux, clasa implementează 7 scenarii și seturi de date de test, acoperind:

1. **Situația normală (scenariul ideal):** Succes la crearea și adăugarea unei activități cu date valide și locuri disponibile.



2. **Limitarea inferioară la zero:** Sistemul respinge crearea unei activități inițializate cu 0 locuri.
3. **Situație anormală (date invalide):** Verificarea comportamentului și respingerea adăugării când numărul de locuri introdus este negativ.
4. **Verificarea unicității ID-ului:** Sistemul blochează duplicarea și nu adaugă o activitate nouă dacă folosește un ID deja existent în listă.
5. **Verificarea unicității titlului:** Sistemul blochează crearea unei activități dacă titlul propus este deja utilizat de o altă activitate din sistem.
6. **Gestionarea erorilor de tip null:** Asigurarea stabilității sistemului prin ignorarea obiectelor nule trimise către metoda de creare.
7. **Testarea inserării multiple (în buclă):** Validarea adăugării succesive a mai multor activități și verificarea corectitudinii dimensiunii listei finale.

2.2 Alta diagramă UML (Diagrama de Deployment)



Descrierea modelului:

Această diagramă de deployment (desfășurare) ilustrează arhitectura fizică a sistemului și modul în care artefactele software sunt distribuite pe nodurile de execuție și de stocare. Diagrama surprinde următoarele componente:

- **Noduri (Nodes) și Artefacte:**

- **PC Utilizator (<<device>>):** Reprezintă stația de lucru a utilizatorului (hardware-ul local) pe care rulează sistemul. În interiorul acestui mediu de execuție se află:
 - **VolunteerManagementApp.jar (<<executable>>):** Fișierul executabil compilat al aplicației Java care gestionează logica ONG-ului.
 - **Colectii Date (RAM) (<<in-memory>>):** Structurile de date stocate temporar în memoria principală a sistemului în timpul rulării aplicației (ex: listele de voluntari, activități, donații).
- **Stocare Locală (<<storage>>):** Reprezintă mediul de stocare fizic (hard drive / SSD) al dispozitivului. Aici este stocat artefactul **Raport.pdf (<<file>>)**, care reprezintă rezultatul generat de aplicație (rapoarte sau adeverințe).
- **GitHub Server (<<server>>):** Reprezintă infrastructura externă de găzduire și control al versiunilor, care conține Cod Sursă (<<source>>) al aplicației.

- **Asocieri / Căi de comunicare:**

- **File System:** Indică interacțiunea locală dintre PC-ul utilizatorului și stocarea locală, ilustrând modul în care aplicația Java scrie/salvează fișierele PDF pe disc.
- **git push/pull:** Reprezintă protocolul de comunicare prin rețea folosit pentru a sincroniza codul sursă de pe PC-ul local cu repository-ul de pe serverul GitHub.

3. Referințe bibliografice

1. Curs și Laborator "Ingineria Sistemelor de Programe", UNSTPB, Facultatea de Automatica și Calculatoare, 2026.
2. Documentația oficială Java (Oracle) - Java Platform, Standard Edition Documentation:
<https://docs.oracle.com/en/java/>
3. Documentația bibliotecii iText pentru generarea rapoartelor PDF în Java:
<https://itextpdf.com/resources>
4. Documentația oficială JUnit 5 (JUnit Jupiter) pentru testarea automatizată a aplicațiilor Java:
<https://junit.org/junit5/docs/current/user-guide/>